

Pointers vs References in C++

Last Updated : 26 Jul, 2025



Prerequisite: [Pointers](#), [References](#)

C and C++ support pointers, which is different from most other programming languages such as Java, Python, Ruby, Perl and PHP as they only support references. But interestingly, C++, along with pointers, also supports references.

On the surface, both references and pointers are very similar as both are used to have one variable provide access to another. With both providing lots of the same capabilities, it's often unclear what is different between these mechanisms. In this article, I will try to illustrate the differences between pointers and references.

Pointers: A pointer is a variable that holds the memory address of another variable. A pointer needs to be dereferenced with the * operator to access the memory location it points to.

References: A reference variable is an alias, that is, another name for an already existing variable. A reference, like a pointer, is also implemented by storing the address of an object.

A reference can be thought of as a constant pointer (not to be confused with a pointer to a constant value!) with automatic indirection, i.e., the compiler will apply the * operator for you.

Understanding the differences between pointers and references is crucial for effective C++ programming. The [C++ Course](#) explains these concepts in depth, helping you choose the right approach in your code.

```
int i = 3;

// A pointer to variable i or "stores the address of i"
int *ptr = &i;

// A reference (or alias) for i.
int &ref = i;
```

Differences:

1. **Initialization:** A pointer can be initialized in this way:

```
int a = 10;
int *p = &a;
// OR
int *p;
p = &a;
```

We can declare and initialize pointer at same step or in multiple line.

2. While in references,

```
int a = 10;
int &p = a; // It is correct
// but
int &p;
p = a; // It is incorrect as we should declare and initialize references at single step
```

NOTE: This difference may vary from compiler to compiler. The above difference is with respect to Turbo IDE.

3. **Reassignment:** A pointer can be re-assigned. This property is useful for the implementation of data structures like a linked list, a tree, etc. See the following example:

```
int a = 5;
int b = 6;
int *p;
p = &a;
p = &b;
```

4. On the other hand, a reference cannot be re-assigned, and must be assigned at initialization.

```
int a = 5;
int b = 6;
int &p = a;
int &p = b; // This will throw an error of "multiple declaration is not allowed"

// However it is valid statement,
int &q = p;
```

5. **Memory Address:** A pointer has its own memory address and size on the stack, whereas a reference shares the same memory address with the original variable and takes up no space on the stack.

```
int &p = a;
cout << &p << endl << &a;
```

6. **NULL value:** A pointer can be assigned NULL directly, whereas a reference cannot be. The constraints associated with references (no NULL, no reassignment) ensure that the underlying operations do not run into an exception situation.

7. **Indirection:** You can have a pointer to pointer (known as a double pointer) offering extra levels of indirection, whereas references only offer one level of indirection. For example,

```
In Pointers,
int a = 10;
int *p;
int **q; // It is valid.
p = &a;
q = &p;

// Whereas in references,
int &p = a;
int &&q = p; // It is reference to reference, so it is an error
```

8. **Arithmetic operations:** Various arithmetic operations can be performed on pointers, whereas there is no such thing called Reference Arithmetic (however, you can perform pointer arithmetic on the address of an object pointed to by a reference, as in `&obj + 5`).

Tabular form of difference between References and Pointers in C++

Aspect	Reference	Pointer
--------	-----------	---------

Definition	A reference is an alias for an existing variable.	A pointer is a variable that stores the memory address of another variable.
Initialization	Must be initialized when declared and cannot be reassigned.	Can be initialized later and can be reassigned to point to different objects.
Nullability	Cannot be null; must always refer to an object.	Can be null, pointing to no object (e.g., nullptr).
Syntax	Uses & for declaration and accessing.	Uses * for declaration and dereferencing, & for address-of.
Dereferencing	No dereferencing required, can be used directly like a normal variable.	Must be dereferenced using * to access the value it points to.
Void Type	A reference can never be void.	A pointer can be declared as void
Nesting	Reference variable has only one/single level of indirection.	A pointer variable has n-levels/multiple levels of indirection i.e. single-pointer, double-pointer, triple-pointer
Pointer Arithmetic	Cannot perform arithmetic operations (e.g., increment or decrement).	Can perform arithmetic operations (e.g., increment or decrement).
Use Case	Primarily used for simpler, more readable references to variables.	Used for more complex memory management, dynamic memory allocation, and handling arrays.
Example	<code>int& ref = x;</code>	<code>int* ptr = &x;</code>

When to use What

The performances are exactly the same as references are implemented internally as pointers. But still, you can keep some points in your mind to decide when to use what:

- Use references:
 - > In function parameters and return types.
- Use pointers:
 - > If pointer arithmetic or passing a NULL pointer is needed. For example, for arrays (Note that accessing an array is implemented using pointer arithmetic).
 - > To implement data structures like a linked list, a tree, etc. and their algorithms. This is so because, in order to point to different cells, we have to use the concept of pointers.

Quoted in C++ FAQ Lite: Use references when you can, and pointers when you have to. References are usually preferred over pointers whenever you don't need "reseating". This usually means that references are most useful in a class's public interface. References typically appear on the skin of an object, and pointers on the inside.

The exception to the above is where a function's parameter or return value needs a "sentinel" reference — a reference that does not refer to an object. This is usually best done by returning/taking a pointer, and giving the "nullptr" value this special significance (references must always alias objects, not a dereferenced null pointer).

Related Article:

[When do we pass arguments as Reference or Pointers?](#)

Article Tags:

C++

cpp-references

cpp-pointer

Explore

C++ Basics



Core Concepts



OOP



Standard Template Library(STL)



Practice & Problems



C++ Courses



Corporate & Communications Address:

A-143, 7th Floor, Sovereign Corporate Tower, Sector- 136, Noida, Uttar Pradesh (201305)

Registered Address:

K 061, Tower K, Gulshan Vivante Apartment, Sector 137, Noida, Gautam Buddh Nagar, Uttar Pradesh, 201305



Company

About Us
Legal
Privacy
Policy
Contact Us
Advertise with us
GFG
Corporate Solution
Training Program

Explore

POTD
Job-A-Thon
Blogs
Nation
Skill Up

Tutorials

Programming Languages
DSA
Web
Technology
AI, ML & Data Science
DevOps
CS Core Subjects
Interview Preparation
Software and Tools

Courses

ML and Data Science
DSA and Placements
Web Development
Programming Languages
DevOps & Cloud
GATE
Trending Technologies

Videos

DSA
Python
Java
C++
Web Development
Data Science
CS Subjects

Preparation Corner

Interview Corner
Aptitude
Puzzles
GfG 160
System Design